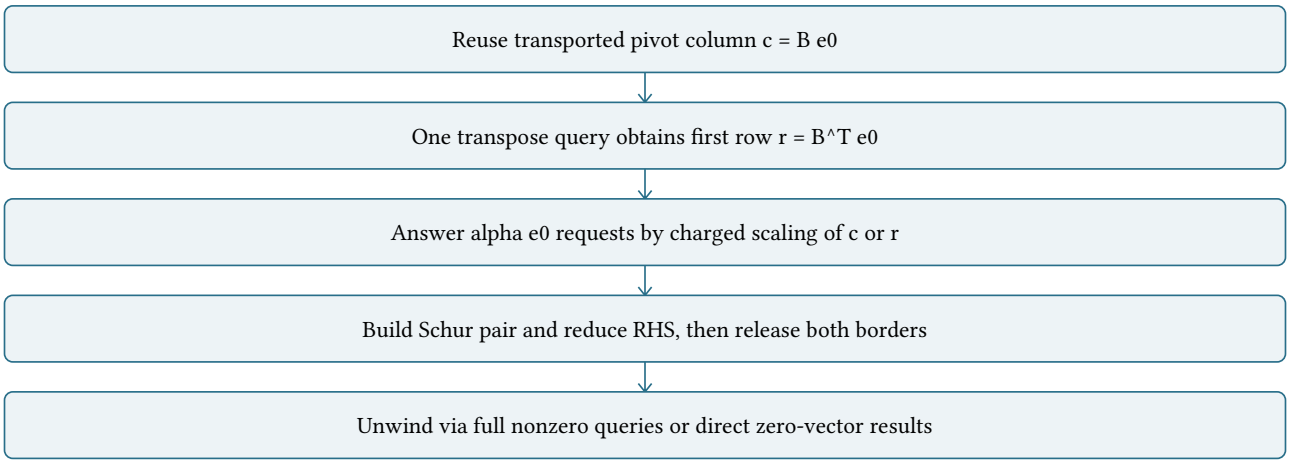
Leading-border reuse in scalar Schur setup

2026-11-02 · Derived schedule and static checks · Numerical trials: zero

Construction. At a scalar-pivot node, the transported search column is already B e0. Obtain B^T e0 with one query. Linearity then gives B(alpha e0)=alpha c and B^T(alpha e0)=alpha r. BorderSchur delegates other inputs to the unchanged source and charges every cached scaling.



Lifetime. Release both borders after generator construction and RHS reduction, before descending. A later lift query has zero first coordinate: if its entire input is zero, return zero directly; otherwise use the source. Nonzero alpha e0 requests after release also delegate, preserving the operator interface. Border storage is O(N) peak instead of retaining a pair at every frame. Other source frames and audit histories can still be large.

Derived per-node counts. Let n≥2, w be the pivoted generator width and ell be one for a nonzero lift input, zero otherwise. Witness lifts have ell=1. Search, pivot transport, compression, child work and root checks are excluded.

Phase	Old queries	New queries	Cache field products
Build first row	0	1	0
Schur generators	2w+6	2	n(2w+4)
Reduce RHS	1	0	n
Unwind lift	1	ell	0
Total	2w+8	3+ell	n(2w+5)

The generator loop has 2w scalar-e0 requests, boundary solves add two, and inner Schur corrections add two. Existing scalar solves and common Schur arithmetic remain charged. Scaling includes zero/one coefficients. Support tests and zero results after release have separate counters.

Native-product connection. Static inspection of the preserved fused kernel gives seven bound products, four native products per generator column and one final scale: 4w+8 instrumented native products per successful application. For the table's phases, counts change from (2w+8)(4w+8) to (3+ell)(4w+8), alongside n(2w+5) additional field products. Do not combine these units at unit price. Operand bit lengths and runtime are unmeasured.

Provenance and checks. The column comes from internal search/transport; canonical entries and corner agreement do not independently certify it. Source generators must remain unchanged. A separately named solver changes only the Schur constructor, profile trace, release call and import; normalized AST comparison passes. A static kernel-schedule checker binds the source hash. No numerical producer, corruption-control or performance trial has run.

Next validation. Full original cohort, both directions, zero/nonzero requests and lifts, width/dimension boundaries, wrong borders, release lifetime, exact ledgers, reference generators, root equations and memory. Feature annihilation remains separate. Existing cost admission still applies.

Sources. border_schur.py, solve_border.py, two static checkers and DERIVATION.md. Bostan et al. (2017), Section 6.2/ Figure 4, DOI 10.1137/16M1062855, supplies established structured Schur context; no randomized or fast matrix-multiplication theorem is imported.